Dr. Dobb's Journal March 2008

Going Superlinear

When it comes to scalability, lots (for lack of a better word) is good

By Herb Sutter

We spend most of our scalability lives inside a triangular box, shown in Figure 1. It reminds me of the early days of flight: We try to lift ourselves away from the rough ground of zero scalability and fly as close as possible to the cloud ceiling of linear speedup. Normally, the Holy Grail of parallel scalability is to linearly use *P* processors or cores to complete some work almost *P* times faster, up to some hopefully high number of cores before our code becomes bound on memory or I/O or something else that means diminishing returns for adding more cores. As Figure 1 illustrates, the traditional shape of our "success" curve lies inside the triangle.

Sometimes, however, we can equip our performance plane with extra tools and safely break through the linear ceiling into the superlinear stratosphere. So the question is: "Under what circumstances can we use *P* cores to do work more than *P* times faster?" There are two main ways to enter that rarefied realm:

- Do disproportionately less work.
- Harness disproportionately more resources.

This month and next, we'll consider situations and techniques that fall into one or both of these categories.

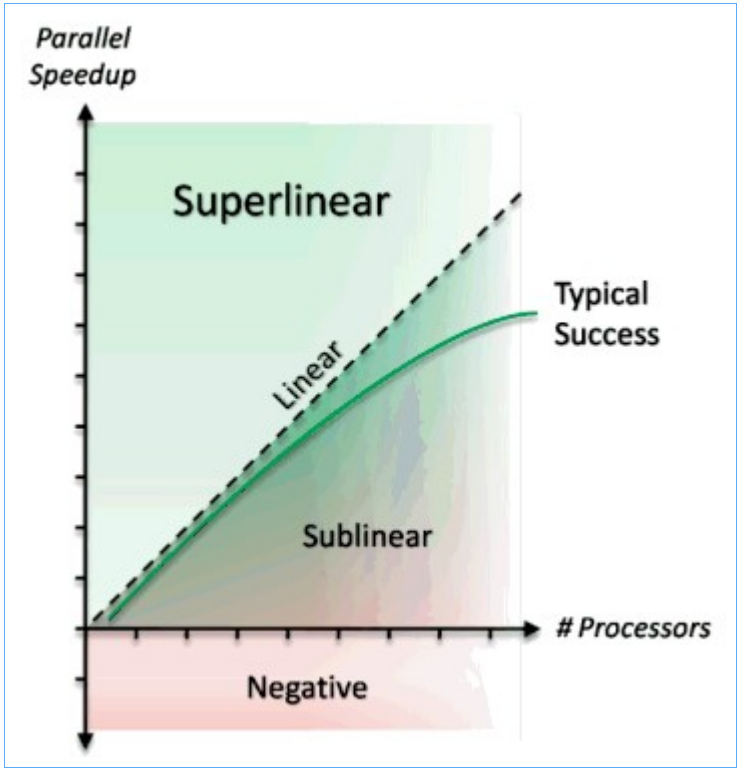

Figure 1: Parallel scalability.

Parallel Search: Exploiting Nonuniformity

Let's say we want to search an unsorted collection of size *N* for a value that matches some criteria. First, consider a simple sequential search:

- If no match exists, the sequential search has no choice but to look at all *N* elements.
- If there's one match, on average, the sequential search has to look at half of the elements to find the match, which is still *O(N)*.
- If there are *M* matches, it typically has to look at proportionately fewer elements, or *O(N/M).*

These results are summarized in the first column of Table 1.

A simple *P*-way parallel search, where each worker explores 1/*P*-th of the collection, will normally be *P* times faster than the sequential search even in the worst case. After all, we're just searching *P* elements at each step instead of only one. (See the second column of Table 1.) Figures 2 and 3 show sample sequential and parallel searches side by side.

[Click image to view at full size]

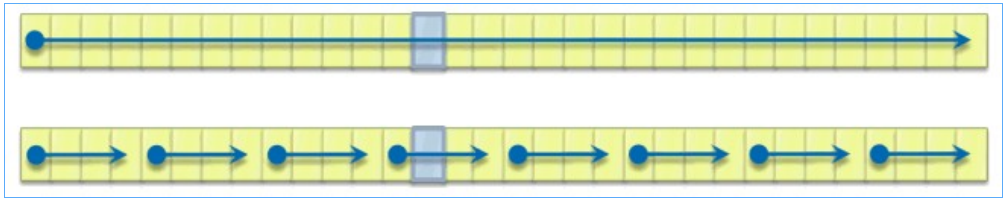

Figure 2: Sequential versus parallel search, single match.

[Click image to view at full size]

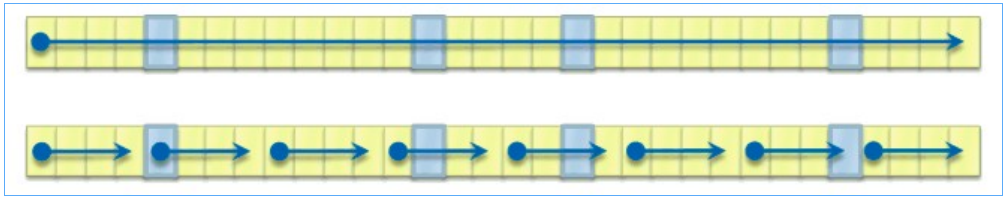

Figure 3: Sequential versus parallel search matches evenly distributed.

[Click image to view at full size]

# Matches	Simple sequential search	Simple P-way parallel search	Parallel speedup
Zero or one matches	$O(N)$	$O(N/P)$	P
M matches, uniformly distributed	$O(N/M)$	$O(N/MP)$	P
M matches, clustered in 1/C-th of the subranges	$O(N/M)$	$O(N/MPC)$	CP

Table 1: Sequential versus parallel search, matches distributed uniformly versus nonuniformly.

But something interesting happens when the distribution of solutions is not uniform, because then averages no longer apply the same way. To illustrate what happens when matches are clustered, consider Figure 4: We still have *M* matches in the collection, as before, but let's assume that they are distributed, not evenly throughout, but with higher probability in some regions.

[Click image to view at full size]

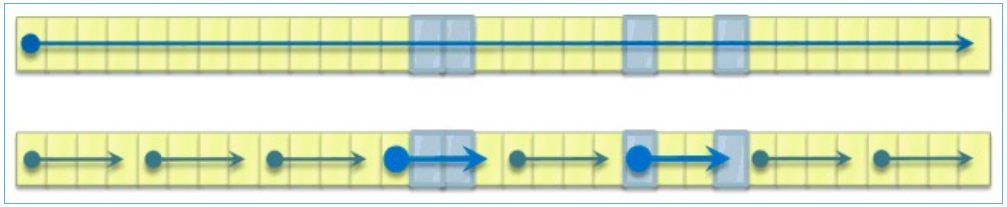

Figure 4: Sequential versus parallel search, when matches are clustered.

For simple sequential search, this is no big news: It fares neither better nor worse, and on average still takes as long to find a match as it would if the matches were distributed uniformly throughout the collection. Even if the sequential algorithm writer knows in advance that matches tend to be clustered, it's hard to see how to take advantage of that without also knowing in advance where the clusters are.

For the simple parallel search, on the other hand, clustering is great news, because a parallel search directly takes advantage of any locality there may be in the matches. Consider: Any clustering effect means that some subrange workers will end up discovering that they are impoverished, having relatively fewer matches, or none at all, in their assigned search areas. But other workers will enjoy a correspondingly higher number of matches in their areas, which means—and this is the important part—they will have a higher probability of finding a match each time they test an element, which in turn means that on average they will need to visit fewer elements to find a match. Like Forty-Niners panning and digging for nonuniformly distributed gold in California (near Sutter's Mill; no relation), many workers will find little or nothing, while a lucky few, or just one, will hit the Motherlode. And "just one" is all it takes; any clustering at all is immediately helpful because the time to perform the total search is the minimum of the workers' runtimes—we're done as soon as any worker finds a match.

The final row of Table 1 illustrates this effect with a simplified example case that defines a clustering factor, *C*: We still have *M* matches, but the matches are concentrated in 1/*C*-th of the subranges (not necessarily contiguous) to be explored by the parallel workers. When *C*=1, we're just in the uniform distribution case. But when *C*>1, the lucky workers' probability of success on each test is higher by a factor of *C*, and because in this example we are guaranteed that one of them will find the answer, this has the same effect on the performance of the entire search as if all workers enjoyed the same boost; that is, it's as if the whole collection contained not just *M* matches, but *CM* matches. Hence the parallel search enjoys not just the expected linear speedup of *P*, but a superlinear speedup of *CP*—the greater the clustering factor, the greater the improvement above and beyond the linear speedup from using a larger number of processors [1].

This isn't limited to simple array-like collections. Many kinds of collections naturally enjoy clustered matches. For example, when we traverse a tree or graph using depth-first search, the parallel version of the algorithm typically has each parallel worker searching a localized subtree, and in many problem domains the solutions tend to be clustered in subtrees. When that happens, one worker will have a disproportionately better chance of being assigned an area with more matches, and we can expect superlinear improvement. Example applications range from searching game trees (for example, a bad move at one node in the tree can cause the entire subtree below it containing subsequent moves to have no good solutions), to CAD and VLSI circuit layout algorithms (see [2]).

A Word About Sequential Efficiency versus Complexity

At this point, it's important to consider and understand a potential objection. "But wait," someone could complain, "your example so far is unfair because you've stacked the deck. The truth is that, when you find a superlinear speedup, what you've really found is an inefficiency in the sequential algorithm." Taking a deep breath, the dissenter might correctly point out: "For any *P*-way parallel algorithm, we can find a sequential algorithm that is only *P* times slower (that is, the parallel advantage is linear, not superlinear) as follows: Just visit the nodes in the same order as the parallel algorithm, only one at a time instead of *P* at a time. Visit the set of nodes visited first by each worker, then the set of nodes visited second by each worker, and so on. Obviously, that will be slower than the parallel version by only a factor of *P*." Give yourself bonus points if you noticed that, and more bonus points if you noticed that even Amdahl's Law (covered last month [3]) implies that the maximum speedup though the use of *P* processors cannot be more than *P*.

It is indeed important and useful to know we can turn any parallel algorithm into a sequential one by just doing the steps of the work one at a time inst of *P* at a time. This is a simple and cool technique that often helps us reason clearly about the performance of our parallel algorithms; we definitely nee

this one in our algorithm design toolchests.

But we cannot therefore conclude that superlinear speedups are simply due to inefficiency in the sequential algorithm ("you should have written the sequential one better"). Let's consider a few pitfalls in the proposed resequentialized algorithm.

First, it has worse memory behavior, especially if (a) the algorithm makes multiple passes over the same region of data and/or (b) the collection is an array. The proposed algorithm has worse memory and cache locality because it deliberately keeps jumping around through the search space. Further, its traversal order is hostile to prefetching: Hardware prefetchers try to hide the cost of accessing memory by noticing that if your program is asking for memory location *X* now, it's likely to want memory locations like *X+1* or *X-1* next, so it can choose to speculatively request those at the same time without actually waiting for the program to ask for them. This typically gives a big performance boost to code that traverses memory in order, either forward or backward, instead of jumping around. (This applies less if the collection is a node-based data structure like a tree or graph, because node-based structures typically make their traversals jump around in memory anyway.)

Note that the parallel algorithm avoids this problem because each worker naturally maintains both linear traversal order and locality within its contiguous subrange. You might expect that if the workers all share the same cache anyway, the analysis could come out the same, but the problem doesn't bite us for reasons we'll consider in more detail next month when we cover hardware considerations. (Hint: The workers typically do not share the same cache...)

Second, it's more complex. It has to do more bookkeeping than a simple linear traversal. This additional work can be a small additional source of performance overhead, but the more important effect is that algorithms that are more complex require more work to write, test, and maintain.

Third, it will sometimes be a pessimization. Even when the values are nonuniform (which is what the complexified algorithm is designed to exploit), sometimes there will be high-probability areas near the front of the collection, and the original sequential search would have searched them thoroughly first whereas the modified version will keep jumping away from them. After all, there's no way to tell what visitation order is best without knowing in advance where the high-probability regions are.

Finally, and perhaps most importantly, when we're comparing the proposed algorithm with simple parallel search, we're not really comparing apples with apples. We are comparing:

- A complex sequential algorithm that has been designed to optimize for certain expected data distributions, and
- A simple parallel algorithm that doesn't make assumptions about distributions, works well for a wide range of distributions, and naturally takes advantage of the special ones the optimized one is trying to exploit...and still gets linear speedup over its optimized sequential competition on the latter's home turf.

On Deck

There are two main ways into the superlinear stratosphere:

- Do disproportionately less work.
- Harness disproportionately more resources.

This month, I focused on the first point. Next month, I conclude that with a few more examples, then consider how to set superlinear speedups by harnessing more resources—quite literally, running on a bigger machine without any change in the hardware.

Acknowledgments

Thanks to Tim Harris, Stephan Lavavej, and Joe Duffy for their input on drafts of this article.

Notes

[1] V.N. Rao and V. Kumar. "Superlinear speedup in parallel state-space search," *Foundations of Software Technology and Theoretical Computer Science* (Springer, 1988).

[2] Si. Pi Ravikumar. *Parallel Methods for VLSI Layout Design*, 4.3.2 (Ablex/Greenwood, 1996).

[3] H. Sutter. "Break Amdahl's Law!" (*DDJ*, February 2008).